فصل ۷: دامنهٔ ویژگی‌های معماری

تفکرِ معمارانِ نرم‌افزار باید هم‌پای اکوسیستمِ ما تکامل یابد و این ضرورت در هیچ جا آشکارتر از تعیینِ دامنهٔ (scope) ویژگی‌های معماری نیست. بسیاری از چارچوب‌های منسوخ برای تعیینِ ویژگی‌های معماری نقصی مهلک داشتند: فرض می‌کردند که کلِ سیستم تنها یک مجموعه ویژگی معماری دارد. هرچند این فرض گاهی هنوز درست است، بسیاری از معماری‌های مدرن، مانند میکروسرویس‌ها، در سطحِ سرویس و در سطحِ سیستم ویژگی‌های معماریِ متفاوتی دارند.

دامنهٔ ویژگی‌های معماری سنجه‌ای سودمند برای معماران است، به‌ویژه برای تعیینِ مناسب‌ترین سبکِ معماری به‌عنوانِ نقطهٔ آغازِ پیاده‌سازی. هنگامِ نوشتنِ کتابِ Building Evolutionary Architectures، به تکنیکی نیاز داشتیم تا تکامل‌پذیریِ ساختاری (structural evolvability) سبک‌های معماریِ مشخص را بسنجیم. هیچ‌یک از سنجه‌های موجود سطحِ جزئیاتِ درستی نداشت. بخشِ «سنجه‌های ساختاری» در صفحهٔ ۸۳ به معیارهای گوناگونی در سطحِ کد می‌پردازد که به معماران امکان می‌دهند جنبه‌های ساختاریِ یک معماری را تحلیل کنند؛ اما هیچ‌یک از این معیارها دامنه را بازتاب نمی‌دهد. این معیارها جزئیاتِ سطحِ پایینِ کد را آشکار می‌کنند، اما نمی‌توانند مؤلفه‌های وابسته‌ای را که بیرون از پایگاه کد قرار دارند (مانند پایگاه‌های داده) و بر بسیاری از ویژگی‌های معماری، به‌ویژه ویژگی‌های عملیاتی، اثر می‌گذارند ارزیابی کنند. معمار هر قدر هم برای طراحیِ پایگاه کدی با کارایی بالا یا کشسان تلاش کند، اگر پایگاه‌دادهٔ سیستم با این ویژگی‌ها هماهنگ نباشد، تلاشش به نتیجه نمی‌رسد.

چون سنجهٔ خوبی برای دامنه نیافتیم، خودمان سنجه‌ای ساختیم. نامش را کوانتوم معماری (architecture quantum) گذاشته‌ایم.

کوانتوم‌های معماری و دانه‌بندی

کوپلینگ در سطحِ مؤلفه تنها چیزی نیست که اجزای نرم‌افزار را به هم پیوند می‌دهد. بسیاری از مفاهیمِ کسب‌وکار بخش‌هایی از سیستم را از نظرِ معنایی به هم پیوند می‌دهند و انسجام کارکردی پدید می‌آورند. معماران و توسعه‌دهندگان برای طراحی، تحلیل و تکاملِ موفقِ نرم‌افزار باید همهٔ نقاط کوپلینگ (coupling points) را که ممکن است دچار شکست شوند در نظر بگیرند.

> **جمعِ اصطلاحاتِ فنیِ لاتینی‌تبار**
>
> واژهٔ `quantum` ریشهٔ لاتینی دارد و از این رو صورتِ جمعِ آن به `a` ختم می‌شود. اگر بیش از یک `quantum` داشته باشید، `quanta` دارید؛ درست مانندِ واژهٔ رایج‌ترِ `data` که آن هم لاتینی‌تبار است. معماران به‌ندرت از یک `datum` به‌تنهایی سخن می‌گویند. راهنمای کاربردِ زبانِ `Chicago Manual of Style` می‌گوید: «اگرچه این واژه در اصل جمعِ `datum` بود، امروزه معمولاً اسمِ غیرشمارش‌پذیر `(mass noun)` به شمار می‌آید و با فعلِ مفرد به کار می‌رود» (ویراستِ هجدهم، ۲۰۲۴، ص. ۳۳۶). (در این ترجمه، صورتِ جمعِ فارسیِ «کوانتوم‌ها» را به کار می‌بریم.)

بسیاری از معمارانِ آشنا با علم مفهومِ کوانتوم را از فیزیک می‌شناسند؛ جایی که به کوچک‌ترین مقدارِ ممکن از چیزی، معمولاً انرژی، اشاره دارد. واژهٔ `quantum` از لاتین می‌آید و به معنای «چه اندازه بزرگ» یا «چه مقدار» است. این واژه در کاربردِ عمومی معمولاً به معنای «چیزی کوچک و بخش‌ناپذیر (تجزیه‌ناپذیر)» است و ما کوانتوم معماری را نیز به همین معنا تعریف می‌کنیم. تعریفِ غیررسمیِ رایجِ دیگری از کوانتوم معماری این است: «کوچک‌ترین بخشِ سیستم که به‌طورِ مستقل اجرا می‌شود.» برای نمونه، میکروسرویس‌ها اغلب کوانتوم‌های معماری را تشکیل می‌دهند و این موضوع به‌خوبی همین تعریف را نشان می‌دهد: یک سرویس می‌تواند، همراه با داده‌ها و دیگر وابستگی‌های خودش، به‌طورِ مستقل درونِ معماری اجرا شود.

کوانتوم معماری دامنهٔ مجموعه‌ای از ویژگی‌های معماری را تعیین می‌کند. ویژگی‌های آن عبارت‌اند از:

- استقرارِ مستقل از دیگر بخش‌های معماری
- انسجام کارکردیِ بالا
- کوپلینگِ ایستای پیاده‌سازیِ بیرونیِ پایین
- ارتباطِ هم‌گام `(synchronous communication)` با دیگر کوانتوم‌ها

این تعریف چند بخش دارد. بیایید آن‌ها را یکی‌یکی باز کنیم:

دامنهٔ مجموعه‌ای از ویژگی‌های معماری را تعیین می‌کند: معماران کوانتوم معماری را به‌عنوانِ مرزی به کار می‌برند که مجموعه‌ای از ویژگی‌های معماری، به‌ویژه ویژگی‌های عملیاتی، را مشخص می‌کند. کوانتوم معماری چون به‌طورِ مستقل استقرارپذیر است و انسجام کارکردیِ بالایی دارد (که در ادامه به آن می‌پردازیم)، سنجهٔ سودمندی برای ماژولاریتیِ معماری فراهم می‌کند.

به‌طورِ مستقل استقرارپذیر `(independently deployable)`: کوانتوم معماری همهٔ مؤلفه‌های لازم برای کارکردِ مستقل از دیگر بخش‌های معماری را در بر دارد. برای نمونه، اگر برنامه‌ای از یک پایگاه‌داده استفاده کند، آن پایگاه‌داده بخشی از کوانتوم است، چون سیستم بدونِ آن کار نمی‌کند. این الزام به آن معناست که تقریباً همهٔ سیستم‌های میراثی `(legacy systems)` که با یک پایگاه‌دادهٔ واحد مستقر می‌شوند، بنا به تعریف، یک کوانتوم واحد تشکیل

می‌دهند. اما در سبکِ معماریِ میکروسرویس‌ها، هر سرویس پایگاه‌دادهٔ خودش را دارد (که بخشی از فلسفهٔ هدایت‌گرِ زمینهٔ محدود است و در فصل ۱۸ به‌تفصیل شرح داده می‌شود). این امر چندین کوانتوم درونِ آن معماری پدید می‌آورد، چون هر سرویس دامنهٔ ویژگی‌های معماریِ خودش را دارد.

انسجام کارکردیِ بالا: انسجام در طراحیِ مؤلفه به این اشاره دارد که کدِ درونِ آن تا چه اندازه در هدفش یکپارچه است. برای نمونه، مؤلفهٔ `Customer` که همهٔ ویژگی‌ها و متدهایش به موجودیتِ `Customer` مربوط‌اند، انسجامِ بالایی دارد. مؤلفهٔ `Utility` با مجموعه‌ای تصادفی از متدهای متفرقه چنین انسجامی ندارد. انسجام کارکردیِ بالا یعنی کوانتوم معماری کاری هدفمند انجام می‌دهد. این تمایز در برنامه‌های تک‌پارچهٔ سنتی با یک پایگاه‌دادهٔ واحد، که انسجام در عمل کلِ سیستم را در بر می‌گیرد، اهمیتِ چندانی ندارد. اما در معماری‌های توزیع‌شده‌ای مانند معماری‌های رویدادمحور `(event-driven)` یا میکروسرویس‌ها، معماران بیشتر هر سرویس را چنان طراحی می‌کنند که با یک گردش‌کارِ واحد (یک زمینهٔ محدود، آن‌گونه که در «زمینهٔ محدود در طراحی حوزه‌محور» شرح داده شده) مطابقت داشته باشد؛ در نتیجه آن سرویس انسجام کارکردیِ بالایی خواهد داشت.

> **زمینهٔ محدود در طراحی حوزه‌محور**
>
> کتابِ `Domain-Driven Design` نوشتهٔ `Eric Evans` (`Addison-Wesley Professional`، ۲۰۰۳) اثرِ ژرفی بر تفکرِ معماریِ مدرن گذاشته است. طراحی حوزه‌محور `(DDD)` یک تکنیکِ مدل‌سازی است که به معماران امکان می‌دهد حوزه‌های مسئلهٔ پیچیده را به‌شکلی سازمان‌یافته تجزیه کنند. `DDD` مفهومِ زمینهٔ محدود را تعریف می‌کند: محدوده‌ای که در آن هر چیزی مربوط به بخشی از حوزه، از درون قابل مشاهده و شفاف است، اما از دیدِ دیگر زمینه‌های محدود مات و پنهان می‌ماند.
>
> پیش از `DDD`، معماران می‌کوشیدند کد را به‌طورِ یکپارچه میانِ موجودیت‌های مشترک در سراسرِ سازمان بازاستفاده کنند. اما دریافتند که ساختنِ مصنوعاتِ مشترک انبوهی از مشکلات پدید می‌آورد؛ مانندِ کوپلینگِ تنگاتنگ، هماهنگیِ دشوارتر و پیچیدگیِ بیشتر. مفهومِ زمینهٔ محدود می‌پذیرد که هر موجودیت در یک زمینهٔ محلی بهترین کارکرد را دارد. از این رو، به‌جای ساختنِ یک کلاسِ `Customer` یکپارچه در سراسرِ سازمان، هر حوزهٔ مسئله می‌تواند کلاسِ `Customer` خودش را بسازد و تفاوت‌ها را در نقاط ارتباطی با دیگر حوزه‌ها تطبیق دهد و حل‌وفصل کند.

برای روشن‌شدنِ بخش‌های بعدیِ تعریف، باید میانِ انواعِ کوپلینگ تمایزهای دقیق‌تری بگذاریم:

کوپلینگِ معنایی `(semantic coupling)`: کوپلینگِ معنایی کوپلینگِ طبیعیِ مسئله‌ای را توصیف می‌کند که معمار برایش راه‌حل می‌سازد. برای نمونه، کوپلینگِ ذاتیِ یک برنامهٔ پردازشِ سفارش چیزهایی مانند موجودیِ انبار، کاتالوگ‌ها، سبدهای خرید، مشتریان و فروش را در بر می‌گیرد. ماهیتِ مسئله‌ای که انگیزهٔ ساختِ یک راه‌حلِ نرم‌افزاری است، این کوپلینگ را تعریف می‌کند. معماران تکنیک‌های اندکی دارند که جلوی موجوار گسترش‌یافتنِ تغییراتِ حوزه در سراسرِ

سیستم را بگیرد: تغییر در حوزه (و در نتیجه در معنا) یعنی الزاماتِ سیستم را تغییر می‌دهیم. هرچند معماران می‌توانند خود را با این وضع سازگار کنند، هیچ الگوی معماریِ جادویی‌ای نمی‌تواند مانع شود که تغییرِ مسئلهٔ اصلی بر معماری اثر بگذارد.

کوپلینگِ پیاده‌سازی `(implementation coupling)`**:** کوپلینگِ پیاده‌سازی توصیف می‌کند که معمار و تیم چگونه تصمیم می‌گیرند وابستگی‌های مشخصی را پیاده کنند. در یک برنامهٔ پردازشِ سفارش، تیم باید برای تعیینِ مرزهای حوزه محدودیت‌های گوناگونی را در نظر بگیرد. برای نمونه، آیا همهٔ داده‌ها باید در یک پایگاه‌دادهٔ واحد قرار گیرند، یا بخشی از آن‌ها برای مقیاس‌پذیری یا در دسترس‌بودنِ بهتر جدا شوند؟ آیا باید یک سیستمِ تک‌پارچه بسازیم یا یک معماری توزیع‌شده؟ پاسخ به این پرسش‌ها اثرِ اندکی بر کوپلینگِ معناییِ سیستم دارد، اما بر تصمیمات معماری اثرِ زیادی می‌گذارد.

کوپلینگِ ایستا `(static coupling)`**:** کوپلینگِ ایستا به «سیم‌کشیِ» یک معماری اشاره دارد؛ یعنی اینکه سرویس‌ها چگونه به یکدیگر وابسته‌اند. دو سرویس اگر هر دو به یک نقطهٔ کوپلینگِ یکسان وابسته باشند، بخشی از یک کوانتوم معماریِ واحدند. برای نمونه، فرض کنید دو میکروسرویس، `Catalog` و `Shipping`، باید اطلاعاتِ نشانی را با هم به اشتراک بگذارند و از این رو هر دو به یک مؤلفهٔ مشترک وابستگی پیدا می‌کنند. چون هر دو سرویس به آن وابستگی کوپل شده‌اند، بخشی از یک کوانتوم معماریِ واحدند.

راهی ساده برای اندیشیدن به کوپلینگ در معماری نرم‌افزار این است: دو چیز با هم کوپل‌اند اگر تغییرِ یکی ممکن است دیگری را بشکند. کوپلینگِ ایستا دامنهٔ وابستگی‌ها را در یک معماری تعریف می‌کند. برای نمونه، اگر چند سرویس از یک پایگاه‌دادهٔ رابطه‌ایِ یکسان استفاده کنند، بخشی از یک کوانتوم‌اند.

کوپلینگِ پویا `(dynamic coupling)`**:** کوپلینگِ پویا نیروهایی را توصیف می‌کند که وقتی کوانتوم‌های معماری باید با یکدیگر ارتباط برقرار کنند، در کارند. برای نمونه، وقتی دو سرویس در حالِ اجرا هستند، باید با هم ارتباط برقرار کنند تا گردش‌کارها را شکل دهند و وظایفی را درونِ سیستم انجام دهند. معماران هنگامِ برقراریِ ارتباط میانِ سرویس‌ها در یک معماری توزیع‌شده باید مصالحه‌ها را در نظر بگیرند؛ موضوعی که در فصل ۱۵ به‌تفصیل بررسی می‌شود.

با در دست داشتنِ این تعریف‌ها، می‌توانیم تعریفِ کوانتوم معماری را کامل کنیم:

کوپلینگِ ایستای پیاده‌سازیِ بیرونیِ پایین: سطحِ کوپلینگِ پیاده‌سازی میانِ کوانتوم‌های معماری باید پایین باشد. این ویژگی نیز از فلسفهٔ `DDD` دربارهٔ کوپلینگِ پایین میانِ زمینه‌های محدود سرچشمه می‌گیرد. کوانتوم‌ها بلوک‌های سازندهٔ عملیاتیِ معماری‌اند که یک سطحِ انتزاع بالاتر از مؤلفه‌ها قرار دارند. این کوانتوم‌ها اغلب با مرزهای سرویس‌ها هم‌پوشانی دارند. این هدف ترجیحِ کلیِ معماران به کوپلینگِ سست `(loose coupling)` میانِ بخش‌های مختلفِ معماری را بازتاب می‌دهد.

به‌طورِ کلی، کوپلینگِ تنگاتنگ `(tight coupling)` زمانی مطلوب است که انسجامِ بالا لازم باشد، مانندِ درونِ یک سرویس یا زیرسیستم. معماری‌ای را «شکننده» می‌نامیم که یک تغییرِ واحد در پیاده‌سازی بتواند در آن اثراتِ جانبیِ موج‌وار و غیرمنتظره‌ای پدید آورد که بسیاری از بخش‌های دیگر را (که ظاهراً بی‌ربط‌اند) دچار شکست کند. هرچه دامنهٔ

کوپلینگِ سیستم گسترده‌تر باشد، کوپلینگِ سست بیشتر کمک می‌کند معماری کمتر شکننده باشد. برای نمونه، معمار ممکن است نامِ یک فیلد را در فراخوانیِ یک سرویس از `State` به `StateCode` تغییر دهد، به گمانِ آنکه فقط یک فراخواننده را تحتِ تأثیر قرار می‌دهد؛ اما ناگهان کشف کند که وابستگی‌های بسیارِ دیگری به‌طورِ غیرمنتظره شکسته‌اند.

> **نکته:** در دامنه‌های محدودتر کوپلینگِ بالاتر مجاز است؛ هرچه دامنه گسترده‌تر باشد، کوپلینگ باید سست‌تر باشد.

ارتباط هم‌گام

ارتباط به کوپلینگ پویا اشاره دارد؛ یعنی زمانی که کوانتوم‌های معماری یکدیگر را فراخوانی می‌کنند، که در معماری‌های توزیع‌شده رایج است. این موضوع به‌ویژه به خانوادهٔ ویژگی‌های عملیاتیِ معماری مربوط است که در بخشِ «ویژگی‌های عملیاتیِ معماری» در صفحهٔ ۵۹ شرح داده شدند، زیرا این ویژگی‌ها اغلب زمان‌بندی و مسدودشدن‌های مهم را در معماری‌های توزیع‌شده تعیین می‌کنند.

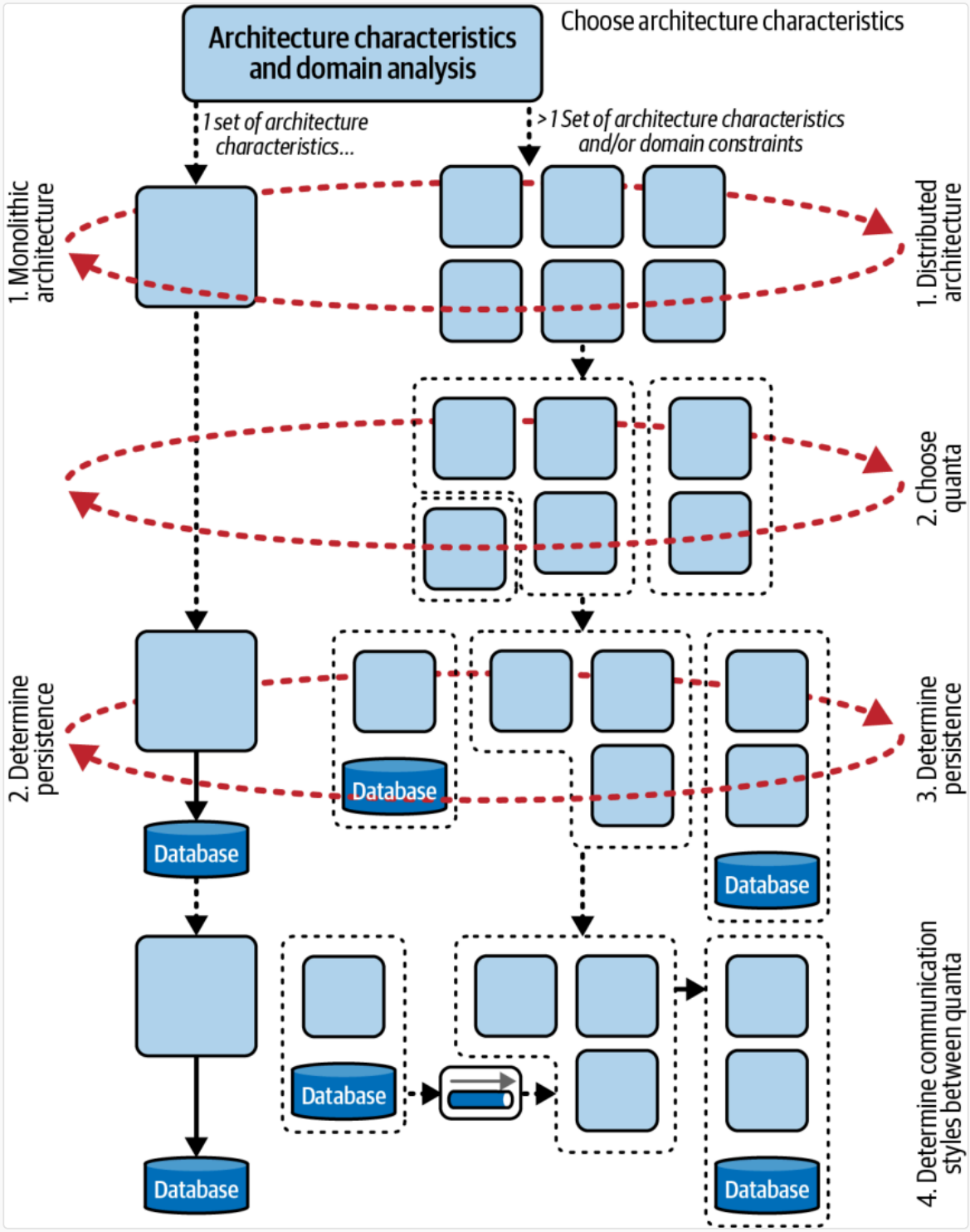
برای نمونه، یک معماری میکروسرویس‌ها را با یک سرویسِ `Payment` و یک سرویسِ `Auction` در نظر بگیرید. وقتی یک حراج به پایان می‌رسد، سرویسِ `Auction` اطلاعاتِ پرداخت را به‌طورِ هم‌گام برای سرویسِ `Payment` می‌فرستد. اما فرض کنید سرویسِ `Payment` فقط می‌تواند هر ۵۰۰ میلی‌ثانیه یک پرداخت را پردازش کند. وقتی شمارِ زیادی از حراج‌ها هم‌زمان پایان یابند، چه رخ می‌دهد؟ این دو سرویس ویژگی‌های عملیاتیِ معماریِ متفاوتی دارند. نخستین فراخوانی موفق خواهد بود، اما پس از آن فراخوانی‌ها یکی پس از دیگری شکست می‌خورند، چون سرویسِ `Payment` از پسِ این شمار درخواست برنمی‌آید؛ مقیاس‌پذیریِ آن به اندازهٔ سرویسِ `Auction` نیست.

اینجا به‌طورِ خاص ارتباطِ هم‌گام را برجسته می‌کنیم، زیرا ارتباطِ ناهم‌گام `(asynchronous communication)` اثرِ بالقوهٔ کمتری دارد. برای نمونه، اگر سرویسِ `Auction` سرویسِ `Payment` را به‌طورِ ناهم‌گام و با استفاده از یک صفِ پیام `(message queue)` فراخوانی کند، صف می‌تواند نقشِ میانگیر `(buffer)` را بازی کند و به هر دو سیستم امکانِ کار بدهد. البته اگر سرویسِ `Auction` پیوسته بیش از ظرفیتِ `Payment` پیام بفرستد، سرانجام صفِ پیام سرریز می‌شود. اما وقتی سیلِ پیام‌ها به‌صورتِ رگباری و مقطعی سرازیر می‌شود، صف می‌تواند پیام‌های معوق را تا زمانی که گیرنده آماده شود نگه دارد. ارتباطِ هم‌گام در معماری‌های توزیع‌شده بسیار انعطاف‌ناپذیر و کم‌تحمل (بی‌گذشت) است، به‌ویژه اگر بخش‌های مختلفِ معماری ویژگی‌های معماریِ متفاوتی داشته باشند. فصل ۱۵ انواعِ ارتباط در معماری‌های رویدادمحور را به‌طورِ کامل بررسی می‌کند.

مفهومِ کوانتوم معماری شیوهٔ تازه‌ای برای اندیشیدن به دامنه فراهم می‌کند. در سیستم‌های مدرن، معماران ویژگی‌های معماری را به‌جای سطحِ سیستم در سطحِ کوانتوم تعریف می‌کنند. این رویکرد هنگامِ تحلیلِ یک حوزهٔ مسئلهٔ جدید اطلاعاتِ مهمی به دست می‌دهد.

اثرِ دامنه‌گذاری (Scoping)

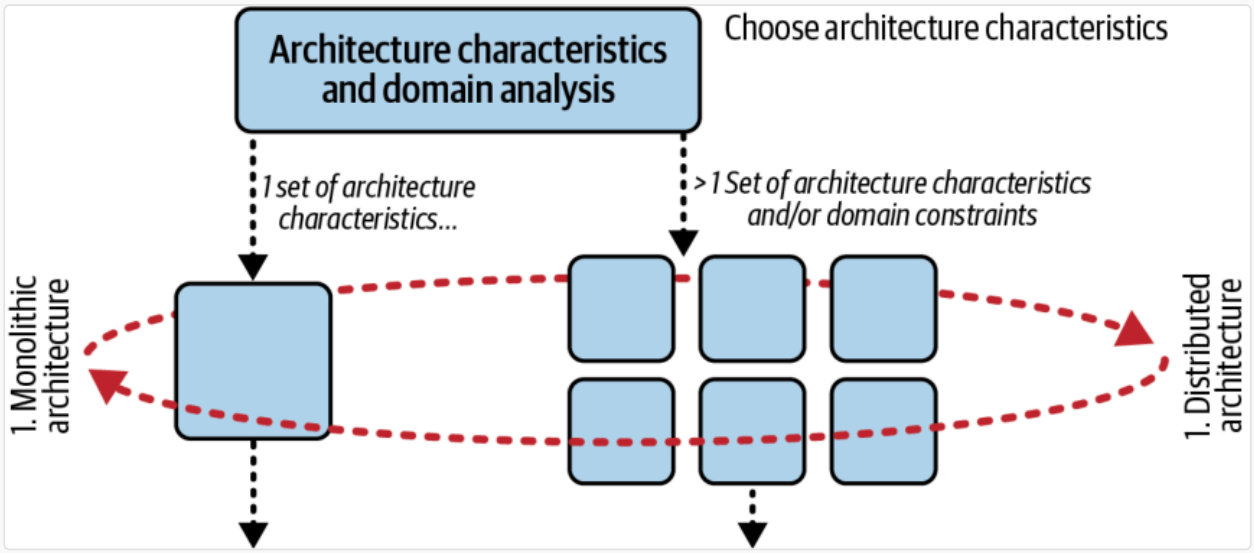
معماران می‌توانند از دامنهٔ ویژگی‌های معماری برای تعیینِ مرزهای مناسبِ سرویس‌ها کمک بگیرند. تصویرِ کلیِ این فرایند در درختِ تصمیمِ (decision tree) شکل ۷-۱ آمده و در گام‌های بعدی با جزئیاتِ بیشتر شرح داده می‌شود.



شکل ۷-۱. درخت تصمیمی که از دامنهٔ ویژگی‌های معماری برای کمک به تعیینِ سبکِ معماری استفاده می‌کند

دامنه‌گذاری و سبکِ معماری

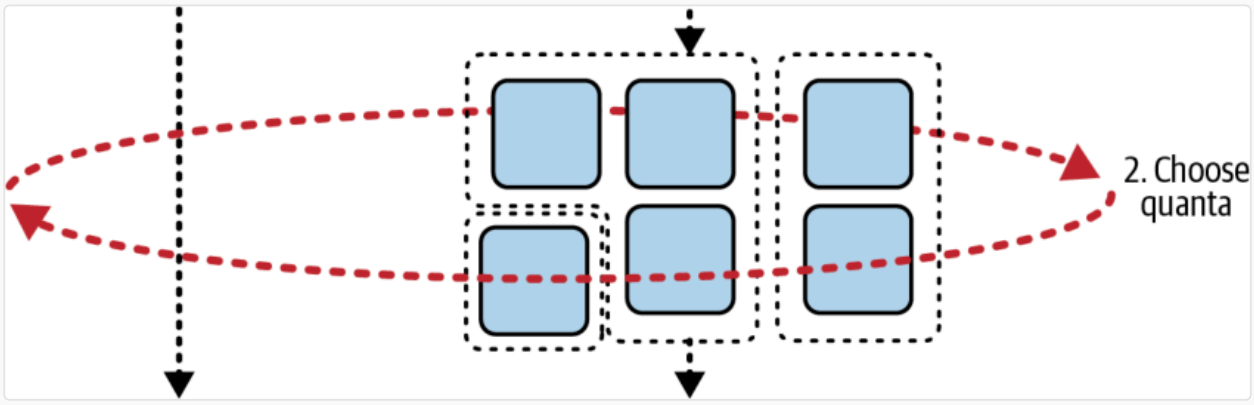
تعیینِ مرزهای کوانتومیِ حوزهٔ مسئله به انتخابِ سبکِ معماری کمک می‌کند: آیا معماری تک‌پارچه مناسب‌تر است یا معماری توزیع‌شده؟ بخشِ نخستِ نمودارِ تصمیمِ نشان‌داده‌شده در شکل ۷-۲ را در نظر بگیرید.



شکل ۷-۲. انتخابِ سبکِ معماریِ مناسب بر پایهٔ ویژگی‌های معماری

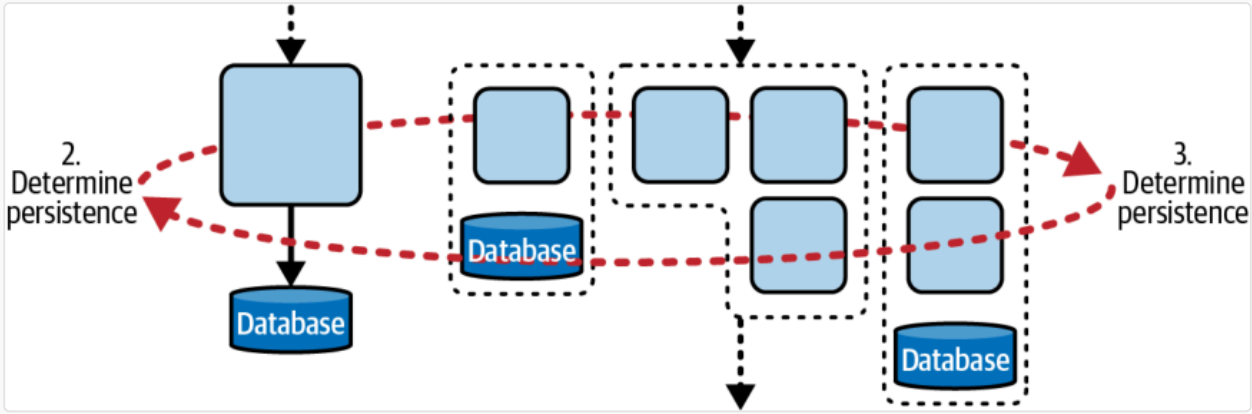
همان‌گونه که در فصل ۴ گفتیم، معماران باید ویژگی‌های معماری و حوزه را تحلیل کنند تا مناسب‌ترین سبکِ معماری را تعیین کنند. بخشی از این تحلیل به این پرسش مربوط است که آیا راه‌حل به چند گروه ویژگی معماری نیاز دارد یا نه. در گامِ نخستِ شکل ۷-۲، معمار تعیین می‌کند که آیا سیستم می‌تواند با یک مجموعهٔ واحد از ویژگی‌های معماری موفق شود یا به بیش از یک گروه نیاز است (برای یک نمونه، بخشِ «کاتا: `Going Green`» در صفحهٔ ۱۰۳ را ببینید).

اگر معمار به این نتیجه برسد که یک مجموعهٔ واحد از ویژگی‌های معماری کافی است، می‌تواند معماری تک‌پارچه را برگزیند و از شمارِ انتخاب‌های بعدی بکاهد. اگر معماری توزیع‌شده را برگزیند، گامِ بعدی تعیینِ مرزهای کوانتومیِ آن است؛ چنان‌که در شکل ۷-۳ نشان داده شده است.



شکل ۷-۳. در معماری‌های توزیع‌شده، معماران باید مرزهای کوانتومیِ مناسب را برگزینند

در فصل ۱۸ رهنمودهایی برای تعیینِ دانه‌بندی ارائه می‌کنیم. گامِ بعدی انتخابِ سازوکارِ ماندگاری (persistence mechanism) است که به هر دو خانوادهٔ معماری مربوط می‌شود و در شکل ۷-۴ نشان داده شده است.



شکل ۷-۴. هر دو سبکِ معماری عموماً به نوعی ماندگاری نیاز دارند

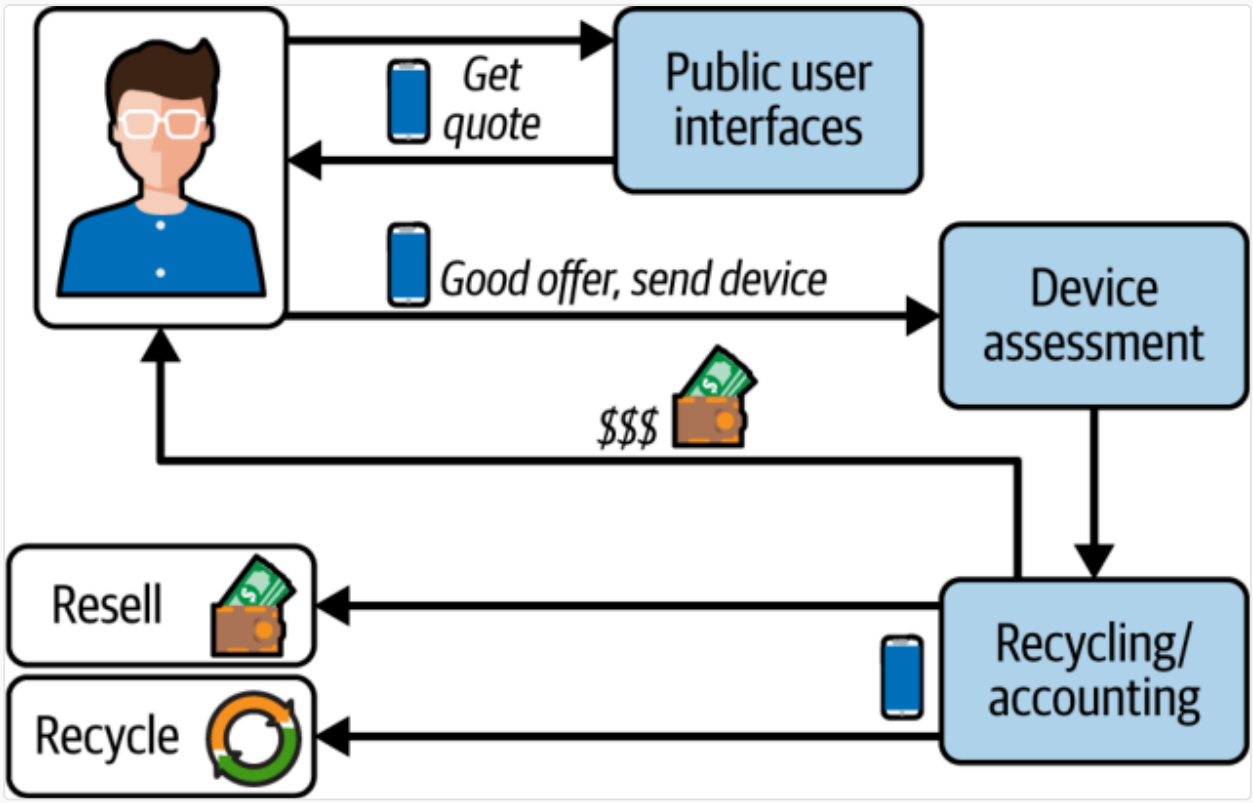
برای معماری‌های تک‌پارچه، یک پایگاه‌دادهٔ تک‌پارچهٔ واحد عموماً مناسب است. معماری و پایگاه‌داده هم‌گام و پابه‌پای هم توسعه و استقرار می‌یابند. فرایند در همین‌جا کامل می‌شود و معمار می‌تواند به انتخابِ مناسب‌ترین سبکِ تک‌پارچه بپردازد.

در مقابل، معماری‌های توزیع‌شده می‌توانند از یک پایگاه‌دادهٔ واحد استفاده کنند (که در معماری‌های رویدادمحور رایج است) یا داده‌ها را در امتدادِ مرزهای دانه‌بندیِ سرویس‌ها افراز کنند (مانندِ معماری‌های میکروسرویس‌ها). یک گامِ دیگر باقی می‌ماند: تعیینِ اینکه میانِ کوانتوم‌ها از چه نوع ارتباطی، هم‌گام یا ناهم‌گام، استفاده شود. (این پرسش را در فصل ۱۵ به‌تفصیل بررسی می‌کنیم.) به یاد داشته باشید که در برخی سیستم‌ها، انتخابِ ارتباطِ هم‌گام می‌تواند مرزهای کوانتومی‌ای را که کوپلینگ ایستا تعیین کرده است تغییر دهد؛ این دو نوع کوپلینگ پیوسته بر یکدیگر اثر می‌گذارند.

کاتا: Going Green

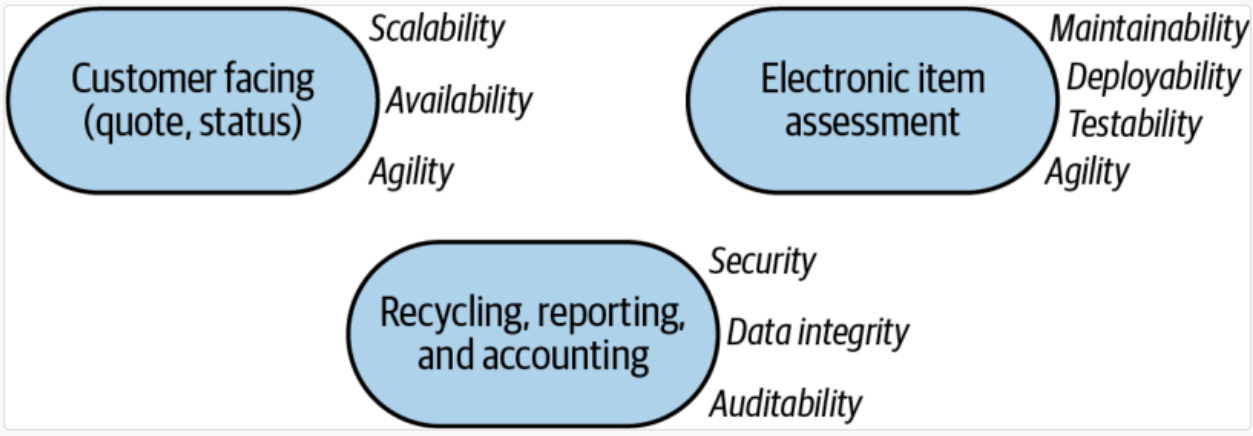
بیایید با به‌کارگیریِ کوانتوم معماری به‌عنوانِ دامنهٔ ویژگی‌های معماری، مسئله‌ای را تحلیل کنیم. این مسئله، با نامِ Going Green، در شکل ۷-۵ نشان داده شده است.

شما با Going Green (GG) کار می‌کنید؛ کسب‌وکاری که لوازمِ الکترونیکیِ کهنه، مانند تلفن‌های همراه، را بازیافت می‌کند و دوباره می‌فروشد. سیستمِ آن هم از کیوسک‌های (kiosks) عمومی و هم از یک وب‌سایت استفاده می‌کند که همگی سیستمِ واحدی را اجرا می‌کنند. کاربر می‌تواند شمارهٔ مدل و وضعیتِ دستگاهش را وارد کند و GG برای آن قیمت پیشنهاد می‌دهد. اگر کاربر پیشنهاد را بپذیرد، می‌تواند دستگاه را در کیوسک تحویل دهد؛ یا اگر از راهِ وب‌سایت اقدام کند، GG جعبه‌ای برایش می‌فرستد تا دستگاه را با پست ارسال کند. GG پس از دریافتِ دستگاه، آن را ارزیابی می‌کند و مبلغ را به کاربر می‌پردازد. سپس ارزشِ دستگاه را برآورد می‌کند و آن را یا بازیافت می‌کند یا دوباره می‌فروشد. سیستمِ GG افزون بر این، گزارش‌ها و دیگر تحلیل‌ها را نیز تولید می‌کند.



شکل ۷-۵. نمای کلیِ الزاماتِ `Going Green`

هنگامِ انجامِ تحلیلِ ویژگی‌های معماری، متوجه می‌شوید که سه خوشهٔ متمایز شکل می‌گیرد؛ چنان‌که در شکل ۷-۶ نشان داده شده است.



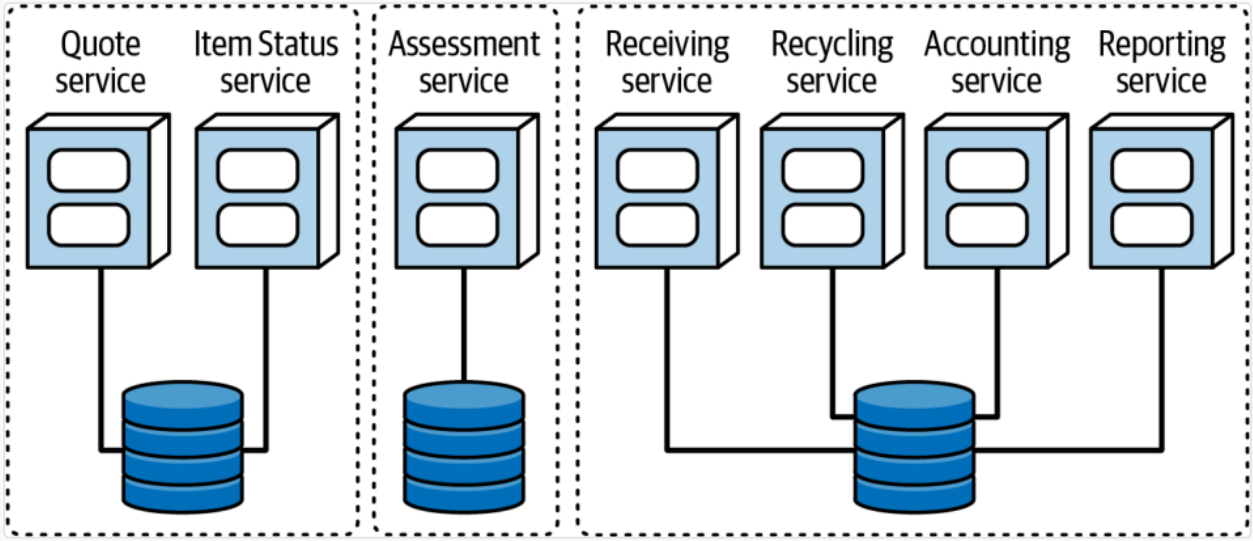
شکل ۷-۶. تحلیلِ ویژگی‌های معماریِ `GG` به سه خوشه از قابلیت‌ها می‌رسد

بخش‌های روبه‌عمومِ `(public-facing)` برنامه به مقیاس‌پذیری، در دسترس‌بودن و چابکی نیاز دارند؛ کارکردهای پشتیبانیِ اداری `(back-office)` به امنیت، یکپارچگی داده و ممیزی‌پذیری نیاز دارند؛ و بخشِ ارزیابی به قابلیت نگهداری، استقرارپذیری و آزمون‌پذیری نیاز دارد (که روی هم ویژگی معماری مرکبِ چابکی را می‌سازند). چرا بخشِ ارزیابی به مجموعهٔ جداگانه‌ای از ویژگی‌های معماری نیاز دارد؟ این نمونهٔ خوبی است از اینکه محرک‌های کسب‌وکار چگونه با

دغدغه‌های معماری تلاقی می‌کنند. مدلِ کسب‌وکارِ GG بر فروشِ دوبارهٔ پرارزش‌ترین لوازمِ الکترونیکیِ دست‌دوم تکیه دارد و مدل‌های جدید پیوسته به بازار می‌آیند. هرچه GG بتواند ارزیابی‌های دستگاه‌هایش را سریع‌تر به‌روز کند، دستگاه‌های جدیدتر (و در نتیجه باارزش‌تری) برای فروشِ دوباره به دست می‌آورد.

آیا می‌توانید سیستمی طراحی کنید که همهٔ این ملاک‌ها را برآورده کند: مقیاس‌پذیری، در دسترس‌بودن، امنیت، یکپارچگی داده، ممیزی‌پذیری، قابلیت نگهداری، استقرارپذیری و آزمون‌پذیری؟ ممکن است... اما دشوار خواهد بود. این ویژگی‌های معماری اغلب اثرِ یکدیگر را خنثی می‌کنند: برای نمونه، دستیابی به استقرارپذیریِ سریع وقتی هم‌زمان دغدغه‌های پشتیبانیِ اداری مانندِ ممیزی‌پذیری را در اولویت گذاشته‌اید دشوارتر است. رابط کاربری نیز به سطحی از مقیاس‌پذیری نیاز دارد که با دیگر بخش‌های سیستم تفاوتِ چشمگیری دارد.

به‌جای آنکه بکوشید همه‌چیز را با هم به دست آورید، می‌توانید خوشه‌های ویژگی‌های معماری را راهنمای جداسازیِ کوانتوم‌ها قرار دهید. در شکل ۷-۷، خط‌چین‌ها هر کوانتوم معماری را نشان می‌دهند. به‌کارگیریِ دامنهٔ ویژگی‌ها به‌عنوانِ راهنمای دانه‌بندیِ سرویس‌ها گامِ نخستِ خوبی در تعیینِ سودمندترین مجموعهٔ مصالحه‌هاست. (این موضوع را در فصل ۱۸ بیشتر بررسی می‌کنیم.)



شکل ۷-۷. این معماریِ GG هر مجموعه از ویژگی‌های معماری را در قالبِ یک مرزِ معماری جای می‌دهد

دامنه‌گذاری و ابر

منابعِ مبتنی بر ابر تصویر را پیچیده‌تر می‌کنند، زیرا بسیاری از ویژگی‌های عملیاتیِ معماریِ سیستم را در خود کپسوله می‌کنند. معماران هنگامِ استفاده از منابعِ مبتنی بر ابر برای تمام یا بخشی از یک برنامه، بسته به مدلِ استقرار، دست‌کم باید دو سناریو را در نظر بگیرند:

استفاده از ابر برای میزبانیِ کانتینرها (containers): بسیاری از تیم‌های توسعه ابر را به‌عنوانِ مرکزدادهٔ جایگزین به کار می‌برند و کانتینرهای سرورها را در آن اجرا (و ارکستراسیون) می‌کنند. در چنین وضعیت‌هایی، معماران باید ویژگی‌های معماریِ کانتینرها و محدودیت‌هایی را که ابزارِ ارکستراسیون (برای نمونه، Kubernetes) تحمیل می‌کند در نظر بگیرند.

استفاده از منابعِ ارائه‌دهندهٔ ابر (cloud provider) به‌عنوانِ مؤلفه‌های سیستم: گونهٔ دیگری از سیستم‌های مبتنی بر ابر برنامه‌ها را با کنار هم گذاشتنِ بلوک‌های سازندهٔ ارائه‌دهندهٔ ابر، مانندِ توابعِ رویدادانگیخته (triggered functions)، پایگاه‌های داده و مانندِ آن، سرهم می‌کند. در این حالت، معماران باید قابلیت‌هایی را که ارائه‌دهنده اعلام می‌کند (و امیدواریم پایبندشان بماند) بررسی کنند تا دریابند چگونه قابلیت‌های مناسبی برای آن زمینهٔ خاص بسازند.

بسیاری از قابلیت‌هایی که امروز آن‌ها را تنظیماتِ پیکربندیِ ارائه‌دهندگانِ ابر می‌شناسیم، مانند کشسانی، با تلاش و مرارتِ فراوان توسطِ نسلِ پیشینِ معمارانی به دست آمده‌اند که روی سیستم‌های فیزیکی کار می‌کردند. دغدغه‌های اصلیِ آنان اکنون آسان‌تر شده است؛ اما ما مجموعهٔ مصالحه‌های مدرنِ خودمان را داریم، مانندِ در دسترس‌بودنِ ارائه‌دهنده و دغدغه‌های امنیتیِ فزاینده. جزئیاتِ معماری نرم‌افزار بسیار تغییر می‌کند، اما کارِ تحلیلِ مصالحه‌ها همواره ثابت می‌ماند.

واژه‌نامهٔ اختصاصی این فصل

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Scope (of Architectural Characteristics)	دامنه (ویژگی‌های معماری)
Structural Evolvability	تکامل‌پذیری ساختاری
Architecture Quantum (pl. Quanta)	کوانتوم معماری (ج. کوانتوم‌ها)
Coupling Point	نقطهٔ کوپلینگ
Independently Deployable	به‌طور مستقل استقرارپذیر
Legacy System	سیستم میراثی
Event-Driven Architecture	معماری رویدادمحور
Semantic Coupling	کوپلینگ معنایی
Implementation Coupling	کوپلینگ پیاده‌سازی
Static Coupling	کوپلینگ ایستا

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Dynamic Coupling	کوپلینگ پویا
Loose Coupling	کوپلینگ سست
Tight Coupling	کوپلینگ تنگاتنگ
Low External Implementation Static Coupling	کوپلینگ ایستای پیاده‌سازی بیرونی پایین
Synchronous Communication	ارتباط هم‌گام
Mass Noun	اسم غیرشمارش‌پذیر
Asynchronous Communication	ارتباط ناهم‌گام
Message Queue	صف پیام
Buffer	میانگیر
Scoping	دامنه‌گذاری
Decision Tree	درخت تصمیم
Persistence Mechanism	سازوکار ماندگاری
Kiosk	کیوسک
Public-Facing	روبه‌عموم
Back-Office	پشتیبانی اداری
Composite Architectural Characteristic	ویژگی معماری مرکب
Container	کانتینر
Orchestration	ارکستراسیون
Cloud Provider	ارائه‌دهندهٔ ابر
Triggered Function	تابع رویدادانگیخته